



FRIDAY • 18:40 • THREE DAYS TO SUBMISSION

Two desks, one deadline. On Friday, Arjun is winning.

COLD OPEN • SCENE 01



MONDAY • 10:15 • THE DEMO MEETS A REAL USER

Monday, somebody clicks a fourth thing.

COLD OPEN • SCENE 02



MONDAY • 10:15 • THE SAME CLICK, ONE DESK OVER

Same click. Same bug. It never happens at her desk.

COLD OPEN • SCENE 03



SAME TOOL • SAME THREE DAYS • SAME MODEL

The difference was never the model.

COLD OPEN • SCENE 04

Microsoft Garage

SkillUp AI

Global Hackathon 2026

From ~~Vibe Coding~~ to Agentic Engineering.

The leap from prompting a toy to **co-architecting** a real codebase.

Tezan Sahu

Applied Scientist II · M365 Copilot Extensibility

A little about me.

Microsoft IDC · 2021 – present

Applied Scientist II — M365 Copilot Extensibility

IIT Bombay · 2017 – 21

Institute Rank 2 · Mechanical Engg (9.90/10) · Minor in CSE.

Writing

Beyond Code (national bestseller) · Low-Pass Filter, a weekly AI newsletter.

Research & teaching

8* US patents · 5 papers · visiting faculty, Bower School of Entrepreneurship.

25,000+ students & professionals trained in AI and Data Science.



li [linkedin.com/in/tezan-sahu](https://www.linkedin.com/in/tezan-sahu)

x [@tezansahu](https://twitter.com/tezansahu)

web tezansahu.com

Vibe coding has a ceiling.

Greenfield demos lie to you. Anything you ship after the demo is brownfield.



Greenfield

WHERE VIBE CODING SHINES

- Empty folder. No conventions to respect, no callers to break.
- One long chat holds the whole project in its head.
- Nothing to regress, so nothing tells you when you regress.

one-shot ✓ ship ✓ wow ✓



Brownfield

WHERE EVERYTHING YOU SHIP ACTUALLY LANDS

- Thousands of files, three frameworks, a module nobody dares touch.
- Decisions that live in people's heads, not in the repo.
- Hidden tests, hidden callers, a CI pipeline with opinions.

hallucinated imports · green locally, red in CI

The fix is not a better prompt. **It is moving your thinking earlier — and writing it down where the agent can read it.**

Every run starts from zero.

The agent is a brilliant new starter who is fired at the end of every shift. Nothing it learnt survives.

// what does this
repo even do?

// what did we agree
yesterday?

// why is it built
this way?

// is this covered
by a test?

// did I already try
this?

⊘ Compacting is not memory

Squeezing the chat into a summary loses fidelity every time — and you can never tell what it dropped.

📄 Files are memory

A spec, a decision record, a ticket. Written once, re-read cheaply, identical on run 1 and run 40.

🔄 So clear early, clear often

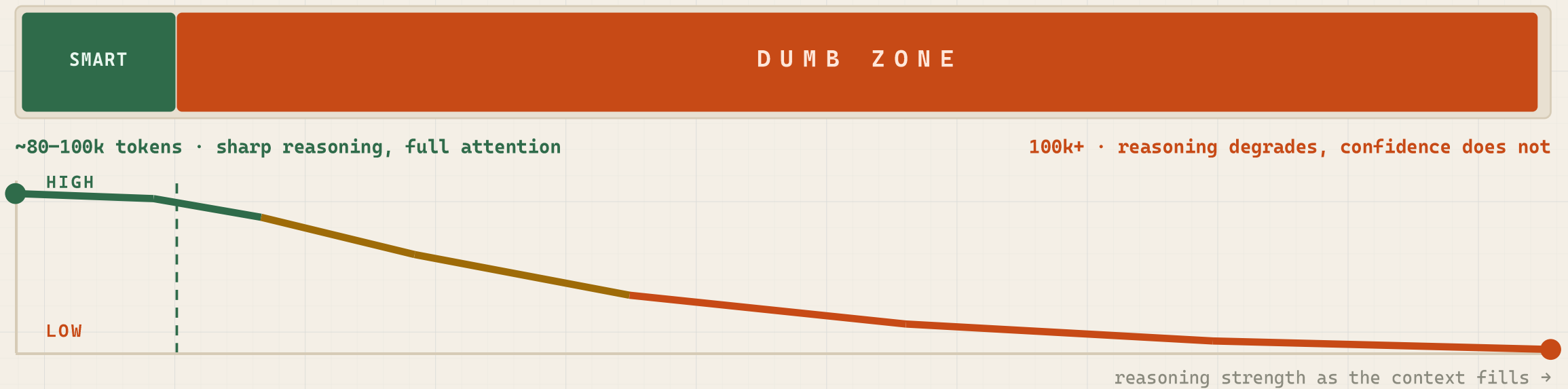
A fresh context that reloads the right three files beats a long one that half-remembers thirty.

Whatever the agent must know has to be on disk before the run begins.

■ FACT TWO ABOUT YOUR COLLABORATOR

1M tokens of context. ~100k tokens of thinking.

Attention is quadratic. Every token you add dilutes the model's focus on every other token.



TRAP

Bigger windows ship more dumb zone. Great for retrieval, bad for reasoning.

TELL

It never announces it. It contradicts an earlier decision and sounds just as sure.

FIX

Size every task to fit inside the smart zone. That is what tickets are for.

■ SO — THE ONLY MOVE THAT SURVIVES BOTH FACTS

Move up a level. You don't type code — you commission it.

BEFORE

you = engineer, typing

ai = autocomplete

Your ceiling is your typing speed.



AFTER

you = architect + PM

ai = staff engineer

Your ceiling is the quality of your decisions.

WHICH MEANS, CONCRETELY



Decide before you build

The expensive mistake is agreeing on the wrong thing, not typing it slowly.



Write the decision down

The artifact — not the chat — is what the next run reads.



Hand execution to the loop

Small, bounded, testable units the machine can drain without you.

The rest of this session is one pipeline that makes all three of those automatic.

GitHub Copilot already ships **eight levers**. Most people touch one.

You opened chat and started typing. Everything below is a knob you own, in your repo, in version control.

model picker

Pick the brain

Swap per task — reasoning, long context, speed.

agent mode

Autonomous loop

Plan → call tools → edit → run → verify → repeat.

plan mode

Read-only think

Drafts a plan, writes nothing. Run it before agent mode.

AGENTS.md

Always-on context

The repo's standing brief. Every tool reads it.

***.instructions.md**

Path-scoped rules

applyTo globs — React rules only for .tsx.

***.prompt.md**

Saved slash commands

One reusable prompt, invoked with arguments.

***.agent.md**

Custom personas

Own model, own tool list, agent-to-agent handoff.

MCP servers

Plug in your systems

Issue tracker, databases, browsers, internal APIs.

Every one of these is a knob, tuned in isolation. **None of them answer: how do I make the agent run this whole workflow, the same way, every time?** → **skills**.

An agent skill is a **folder of markdown** that teaches the agent one job.

Not a fact. A procedure — the steps, the house rules, the references to pull, the scripts to run. Loaded only when it is needed.

ANATOMY

```
.github/skills/  
└─ release-notes/  
    └─ SKILL.md      the steps + when to fire  
    └─ references/    docs it pulls on demand  
    └─ scripts/      code it runs, not reasons about
```

Only **name** and **description** sit in context. The body is read the moment the agent decides the skill applies — so a hundred skills cost you almost nothing.

A SKILL YOU COULD WRITE TODAY

/release-notes

Read every commit since the last tag. Group them the way this team groups them. Write in the house voice from **references/VOICE.md**. Then run **scripts/changelog.py** and open the PR.

Write it once. It is right on Monday, right at 2 a.m., right for the teammate who has never done it — and it is in the repo, so it is reviewed like code.

A skill is not a prompt

A prompt is one shot. A skill is a path the agent walks every time.

Length is not the point

Some of the best skills are four lines. Constraint beats verbosity.

You own them

Markdown in your repo — read them, diff them, fix them, share them.

THE MAP FOR THE NEXT 30 MINUTES

Six skills. Three you drive, three the loop drives.

Everything from here is a live walk through this pipeline in a real repo you can clone.

YOU INVOKE THESE — HUMAN IN THE LOOP

- 01 **/grill-with-docs** Interrogates you until the design holds up. Writes context + decision records.
- 02 **/to-spec** Folds the agreed design into a standalone spec — seams included.
- 03 **/to-tickets** Cuts the spec into tracer-bullet tickets with blocking edges.

THE RALPH LOOP INVOKES THESE

- /implement**
orchestrates the slice
- /tdd**
red → green, at agreed seams
- /code-review**
standards || spec, in parallel

AND THE ARTIFACTS THAT SURVIVE THE RUN

CONTEXT.md

the domain, in your words

docs/adr/

why, not what

docs/spec.md

the agreed design

issues/

one slice per file

git log

decisions, commit by commit

/grill-with-docs

A relentless interview that stress-tests your idea against the repo — before a single line is written.

READS

the feature brief
the existing codebase
CONTEXT.md, docs/adr/

WRITES

a sharpened design
CONTEXT.md (glossary)
docs/adr/000N-*.md

IN THE DEMO

I answer live, off the cuff
you vote on one question
watch it get harder

Say the target folder out loud when it offers to externalise — otherwise it guesses, and every downstream skill points at the wrong docs.

Every answer **prunes a branch**. The leaf is the design.

A design starts as a fat trunk of possibility. Grilling walks you down it, one answer at a time, until one branch is left.

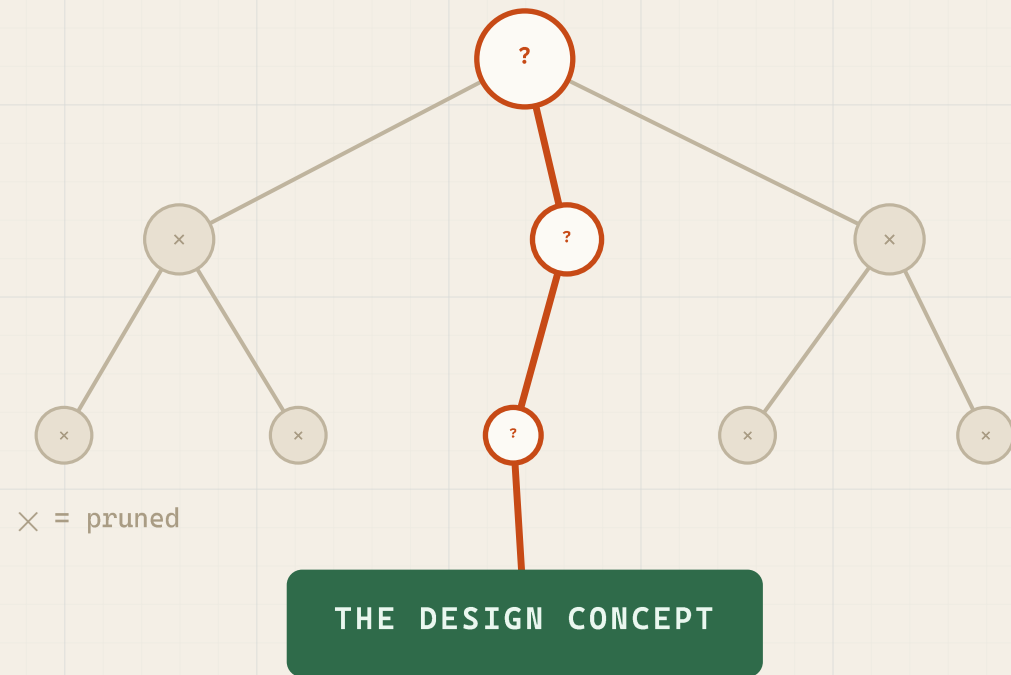
QUESTIONS IT ACTUALLY ASKED

- "What makes two things correlated here — and what is just noise?"
- "How many days of data before you are willing to show a claim?"
- "What must this never say out loud about a person?"
- "When the data is stale, do you hide the insight or caveat it?"

10 to 20 questions is normal.

~50 happens on a hard feature.

That is not the skill failing — that is the design space actually being explored.



what you and the agent both believe you are building

■ THE QUESTION YOU ARE ABOUT TO ASK ME

"Isn't this just plan mode with extra steps?"

No — the arrow points the other way, and that single inversion is the whole trick.

PLAN MODE

AI → you

"Here is my plan. Approve?"

The agent guesses what you want, writes it down confidently, and asks you to rubber-stamp its guess. You review a document you did not shape.

VS

GRILLING

you → AI

"What must this never auto-approve?"

The agent extracts what you already know but never wrote down. You stay the source of taste, and the design is yours by construction.

🔍 Grounded, not abstract

It explores the repo as it asks, so the questions are about your code, not a generic app.

👋 You are the domain expert

The agent's taste is, charitably, average. Yours is the only taste in the room that matters.

📄 The transcript is the asset

The conversation itself is the design record — the next two skills just crystallise it.

An ADR records **why** — so nobody “fixes” it back.

Architecture Decision Record: one to three sentences, numbered, in docs/adr/. Code says what. Only this says why.

```
docs/adr/0003-compute-insights-on-read.md

# Insights are computed on read, not stored

A correlation is cheap to compute and its meaning
changes the moment a new entry lands. Caching it
would create a second source of truth we would
then have to invalidate. We compute per request
and revisit only if p95 crosses 200 ms.
```

THAT IS A COMPLETE ADR

No template, no status field, no considered-options table — unless a rejected option is genuinely worth remembering. The value is that the decision exists and is dated, not that the form is complete.

OFFER AN ADR ONLY WHEN ALL THREE ARE TRUE

- 1 Hard to reverse**
Changing your mind later actually costs something.
- 2 Surprising without context**
A future reader will look at the code and ask, “*why on earth?*”
- 3 A real trade-off**
There were genuine alternatives and you picked one for reasons.

Easy to reverse? Skip it — you will just reverse it.
No real alternative? There is nothing to record beyond “we did the obvious thing.”

/to-spec

No interview. It just crystallises the conversation you already had into one standalone document.

READS

the grilling transcript
the codebase
CONTEXT.md, docs/adr/

WRITES

docs/spec.md
the seams under test
a long list of user stories

IN THE DEMO

watch it propose seams
I push back on one
nothing new is invented

The spec is standalone — it never cites an ADR number. Someone should be able to read it cold and know exactly what to build.

The spec is your externalised brain.

Conversation → document. If your session dies tonight, a fresh agent reads this tomorrow and knows what you agreed.

Problem Statement

The problem, from the user's point of view.
Not the solution wearing a disguise.

Solution

What changes for that user. Still their
language, not yours.

User Stories

A long, numbered list. As an <actor>, I want
<feature>, so that <benefit>. Exhaustive on
purpose.

Implementation Decisions

Modules touched, interfaces changed,
schema, API contracts. No file paths — they go
stale in a day.

Testing Decisions

The seams under test and what a good test
looks like here. Prior art from this repo.

Out of Scope

The quiet hero. It is the agent's hard definition
of done, and it kills scope creep before it
starts.

SEAMS — WHERE THE TESTS WILL LIVE

Prefer an existing seam. Take the highest one you can.
The ideal number of new seams is one — and the skill checks that
with you before it writes a word.

AND NO, I DO NOT READ IT LINE BY LINE

The design is already in my head — I just spent ninety minutes
building it. The spec is written for the next amnesiac run, not for
me. Review happens against running code.

/to-tickets

03

Cuts the spec into tracer-bullet tickets — each a thin vertical slice, each declaring what blocks it.

READS

docs/spec.md
the codebase
the domain glossary

WRITES

issues/01-*.md ... NN
blocking edges
status: ready-for-agent

IN THE DEMO

the first cut is horizontal
I push back, live
we re-cut it vertically

This is the one output you must actually read. The tickets are the instructions to the machine — everything after this is the machine following them.

Don't build a layer. Fire one bullet.

Anti-aircraft gunners load one glowing round in every six — not for the damage, for the feedback. Same idea, same reason.

HORIZONTAL · one layer at a time

UI	all six cards
API	every endpoint
LOGIC	twelve correlations
STORAGE	the whole new schema
TESTS	written last, or never

↳ first feedback arrives at 1,000 lines

VERTICAL · one tracer at a time

UI	1 card	then the next five
API	1 endpoint	then the next four
LOGIC	1 correlation	then the next eleven
STORAGE	1 field	then the rest
TESTS	1 test	then the edge cases

↳ end-to-end on day one, then expand

Wide refactors are the exception.

A rename with a huge blast radius cannot land green as one slice — sequence it expand → migrate in batches → contract instead.

Every ticket declares what blocks it.

A sequential plan can only be picked up by one agent. A graph with blocking edges has a frontier — and a frontier can be drained in parallel.



LABELS THAT DECIDE WHO DOES THE WORK

afk

The machine can do this alone. Implementation, mostly.

hitl

A human must sit here. Design, judgement, anything irreversible.

ready-for-agent

You have read it and you agree it is a correct slice.

/implement

04

The only skill you never type. The loop calls it, once per ticket, in a brand-new context every time.

READS

one ticket
the spec + ADRs
the last five commits

WRITES

a passing vertical slice
tests at agreed seams
a decision-rich commit

IN THE DEMO

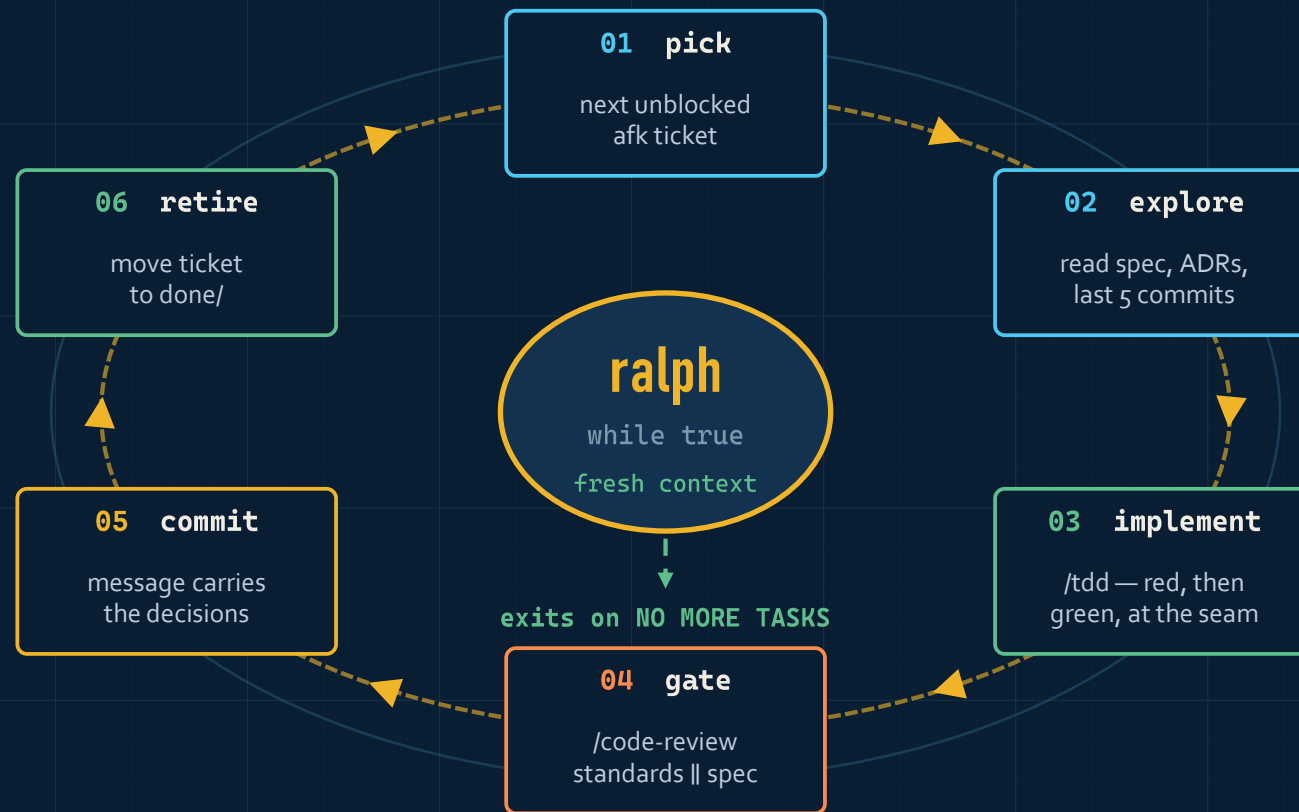
I kick off the loop
we let it run
we come back to it later

It orchestrates: /tdd at the agreed seams, then /code-review as a gate, then commit, then move the ticket to done. No questions asked, no subagents, one task only.

THE DUMB-PHASE ENGINE · IT IS GENUINELY JUST A WHILE LOOP

Meet Ralph.

Named after the Simpsons kid whose entire contribution is "I'm helping!".
Set expectations accordingly — then read the commits.



"I'm helping!"



once.ps1

one ticket, you watch · Use this until you trust the loop.

afk.ps1 -Iterations 20

N tickets, you leave · Exits early on NO MORE TASKS.

powershell

```
$ ./ralph/afk.ps1 -Iterations 20
> iter 1 · #01 tracer · red → green
> review clean · committed · done/
> iter 2 · #02 · red → green
> review: 1 smell · fixed · committed
> iter 7 · NO MORE TASKS · exit 0
```

Every iteration starts clean — so every iteration is in the smart zone.

■ THE FIRST SKILL /IMPLEMENT REACHES FOR

/tdd — because a passing test is the only thing the loop can trust.

If your codebase has no feedback loops, no amount of model quality saves you. The feedback loop is the ceiling.

THE LOOP, PER SLICE

- 1 RED** write the failing test first
- 2 GREEN** only enough code to pass it
- 3 NEXT** one seam, one test, repeat

*Refactoring is deliberately NOT in this loop
It belongs to review.*

SEAMS

The public boundary you observe behaviour at, without reaching inside. No test is written at a seam you have not agreed to — that is how testing effort lands on the critical path, not every edge case.

THE THREE WAYS AGENT-WRITTEN TESTS LIE TO YOU

Implementation-coupled

Mocks internal collaborators, tests private methods, or checks the database instead of the interface.

The tell: refactor changes nothing about behaviour, and the suite goes red.

Tautological

The assertion recomputes the expected value exactly the way the code does.
`expect(add(a,b)).toBe(a+b).`

It passes by construction, so it can never disagree with the code. Expected values must come from outside.

Horizontal slicing

All the tests first, then all the implementation — so the tests describe imagined behaviour, not real behaviour.

Work in vertical slices: one test, one implementation, then let what you learnt shape the next.

THE GATE BEFORE EVERY COMMIT

/code-review — two axes, two sub-agents, never merged.

A change can pass one axis and fail the other. Report them together and one quietly masks the other.

```
git diff <fixed-point>...HEAD
```

STANDARDS

Does the code follow the way this repo writes code?

- Documented repo standards win, always.
- Plus a fixed baseline of Fowler smells.
- Hard violations vs judgement calls, marked.

SPEC

Does the code do what the ticket actually asked?

- Requirements missing or only half-done.
- Behaviour nobody asked for — scope creep.
- Looks implemented, but implemented wrong.

run in parallel · isolated contexts · reported side by side, never reranked

THE SMELL BASELINE — FOWLER, REFACTORING CH.3

Carried even when the repo documents nothing. Every one is a labelled heuristic, never a hard violation.

Mysterious Name

no honest name = murky design

Feature Envy

reaches into another object's data

Primitive Obsession

a string standing in for a type

Shotgun Surgery

one change, edits scattered everywhere

Speculative Generality

hooks for needs the spec does not have

Middle Man

a class that only delegates onward

Duplicated Code

same shape in two hunks

Data Clumps

the same fields always travel together

Repeated Switches

the same cascade on the same type

Divergent Change

one file edited for unrelated reasons

Message Chains

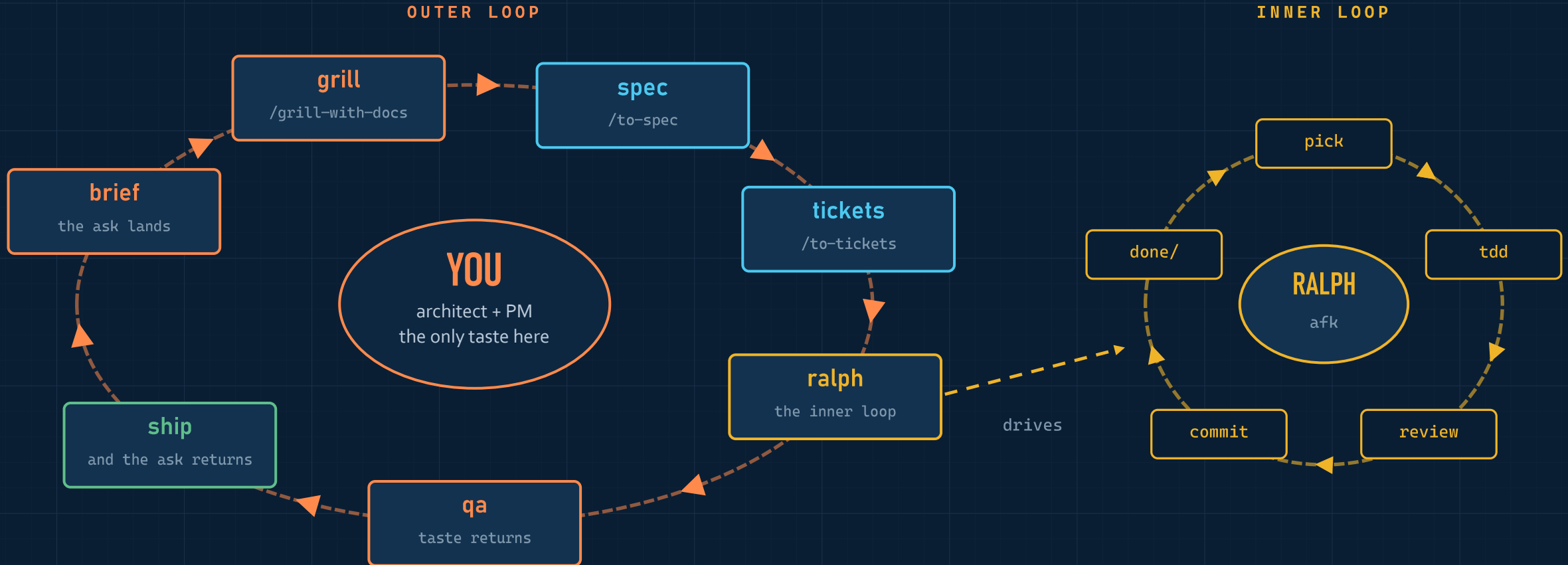
a.b().c().d() the caller shouldn't know

Refused Bequest

a subclass ignoring what it inherits

The agentic engineering loop.

You turn the outer wheel. Ralph turns the inner one. Shipping is not the end of the line — it is what starts the next revolution.



● you steer · taste

● artifacts · contracts

● the machine · afk

● verified · shipped

Steal the pipeline. **Not the tips.**

Three days is exactly the horizon where this pays off. You have time to design once and let the machine build twice.

HOURL 0

Clone the companion repo

Copy .agents/skills/ and .ralph/ into your hack repo. Point VS Code at the skills folder. Two minutes.

HOURL 0-2

Grill before you build

Run /grill-with-docs on your idea. Tell it where to write CONTEXT.md and the ADRs. Answer honestly — including “I don't know yet”.

HOURL 2-3

Spec, then cut

/to-spec, then /to-tickets. Reject the first ticket split if it is horizontal. Make it re-cut vertically.

OVERNIGHT

Let the loop drain it

Fill the ralph prompt template in the same session that did the design — it already knows your paths. Then afk.ps1 and go to sleep.

**EVERY
MORNING**

QA, then re-cut

Read the commits, not the diffs. Whatever QA finds becomes tomorrow night's tickets.

House rule for your team: **no code until somebody can explain the design without opening the editor.**

■ ONE LINE TO TAKE WITH YOU

Stop vibing. Start engineering.

AI did not invent any of these ideas. It made them non-optional.

THE THREE BOOKS UNDER ALL OF THIS

Brooks · *The Design of Design*

the design concept · the tree

Hunt & Thomas · *The Pragmatic Programmer*

tracer bullets · vertical slices

Fowler · *Refactoring*

the smell baseline in code-review

The companion repo: [microsoft-idc-agentic-engineering-workshop-2026](#)

THANK YOU



connect on LinkedIn

linkedin.com/in/tezan-sahu



read Low-Pass Filter

lowpassfilter.substack.com

Questions — and if you build something with this at the hackathon, send it to me.